

TP2 – Transformée de Fourier et traitement du signal

HAMLIL Mohamed

2024-09-01

Table des matières

1	TP2 – Transformée de Fourier et traitement du signal	1
1.1	4. Transformée de Fourier	1
1.2	5.1 Échantillonnage	2
1.3	5.5 Fast Fourier Transform (FFT)	3
1.4	5.6 Filtrage du signal	4

1 TP2 – Transformée de Fourier et traitement du signal

Ce notebook contient des extraits de code relatifs au calcul de transformées de Fourier et au traitement de signaux discrétisés. Les exemples sont directement tirés du document et accompagnés de commentaires pour vous guider.

1.1 4. Transformée de Fourier

Dans cette première partie, nous calculons la transformée de Fourier d’une fonction indicatrice.

```
# Définition de la fonction f(x) = 1 lorsque |x| < 1, sinon 0
f <- function(x) { as.numeric(abs(x) < 1) }

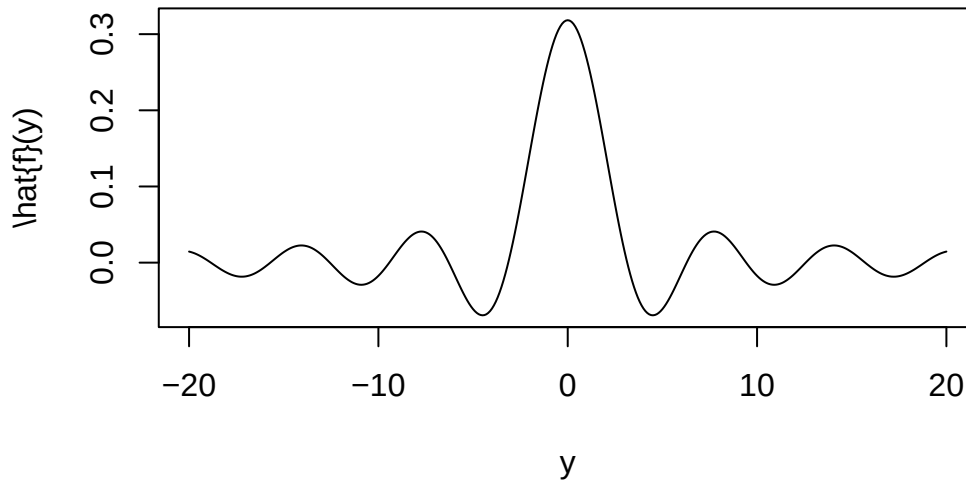
# Fonction e(x, y) = exp(i * x * y)
e <- function(x, y) { exp(1i * x * y) }

# Transformée de Fourier \hat{f}(y)
tf <- function(y) {
  g <- function(x) { f(x) * e(x, y) } # produit f(x) * exp(i*x*y)
  reg <- function(x) { Re(g(x)) }     # partie réelle
  img <- function(x) { Im(g(x)) }     # partie imaginaire
  # Intégrale sur un intervalle large pour approcher la transformée
  (1 / (2 * pi)) * (integrate(reg, -100, 100)$value + 1i * integrate(img, -100, 100)$value)
}

# Discrétisation de l’axe des fréquences
discr <- seq(-20, 20, 0.1)

# Calculer la transformée pour chaque point de discr
ltf <- discr
for (i in 1:length(discr)) {
  ltf[i] <- tf(discr[i])
}
```

```
# Tracer la transformée de Fourier
plot(discr, ltf, type = "l", ylab = "\\hat{f}(y)", xlab = "y")
```



1.2 5.1 Échantillonnage

Nous allons créer un signal analogique composé de plusieurs sinusoïdes et l'échantillonner à une fréquence fixée.

```
# Discrétisation du temps pour l'échantillonnage
discr <- seq(0, 2, 0.125)      # points d'échantillonnage toutes les 0,125 ms (8 kHz)

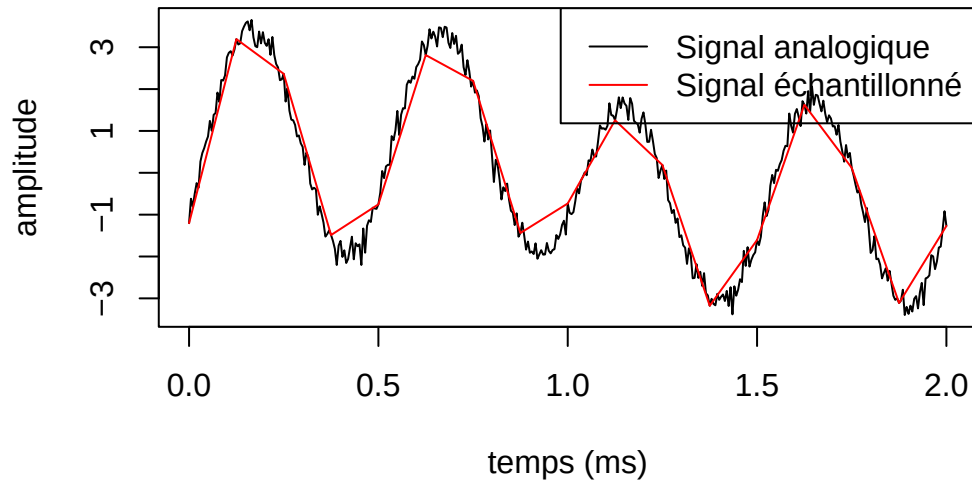
# Discrétisation fine pour le signal analogique
discrn <- seq(0, 2, 0.005)     # pas fin pour le signal de référence

# Construction d'un signal naturel (analogique) avec un peu de bruit
signaln <- sin(discrn * pi) + 0.5 * sin(discrn * 3 * pi) - 2.5 * sin(discrn * 4 * pi + 9) + 0.2 * rnorm(
length(discrn))

# Échantillonnage du signal : on prend une valeur toutes les 25 valeurs du signal analogique
signale <- signaln[seq(1, length(signaln), 25)]

# Tracer le signal analogique et le signal échantillonné
plot(discrn, signaln, type = "l", xlab = "temps (ms)", ylab = "amplitude", main = "Signal analogique et
échantillonné", col = "black", lty = 1)
lines(discr, signale, col = "red", lty = 1)
legend("topright", legend = c("Signal analogique", "Signal échantillonné"), col = c("black", "red"), lty = c(1, 1))
```

Signal analogique et échantillonné



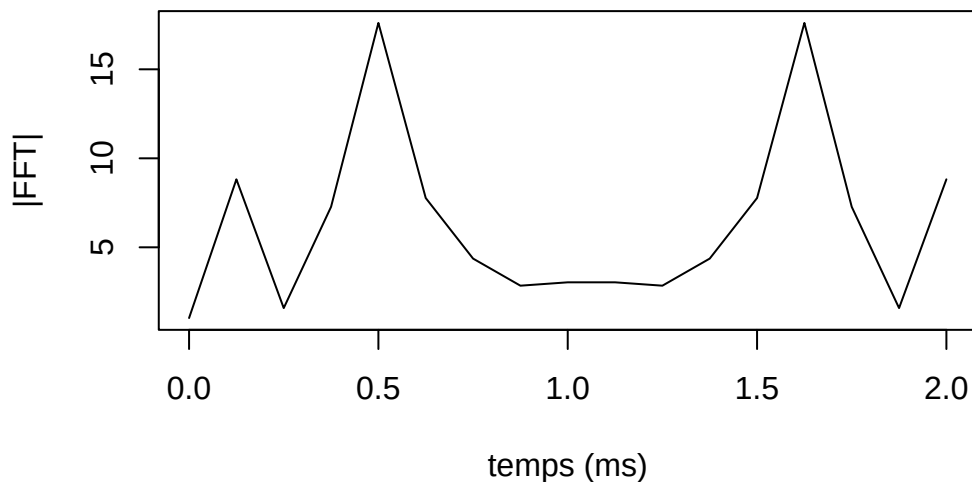
1.3 5.5 Fast Fourier Transform (FFT)

Nous appliquons la FFT au signal échantillonné pour obtenir les coefficients de Fourier et visualiser le spectre de fréquences.

```
# Calcul de la FFT sur le signal échantillonné
fftc <- fft(signale)

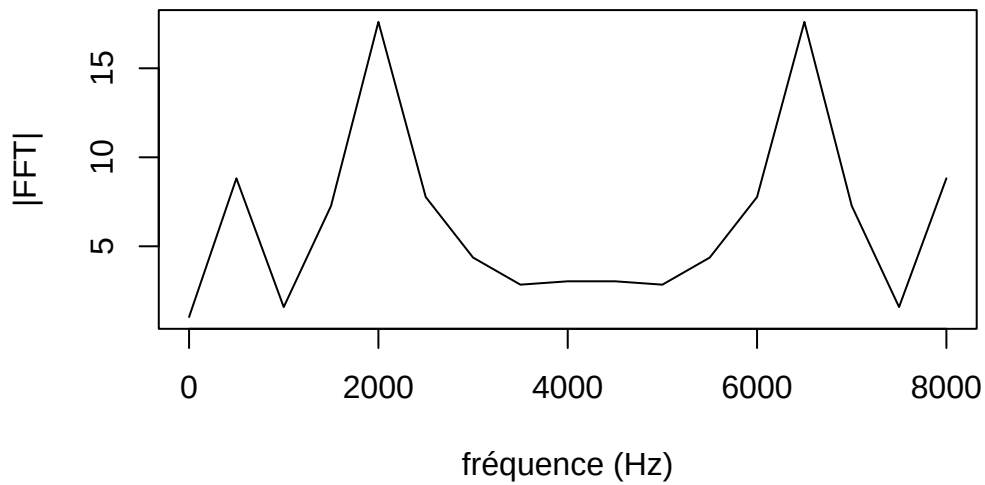
# Le résultat de fft() est un vecteur complexe ;
# nous traçons le module (magnitudes) pour voir quelles fréquences sont présentes.
# Premier tracé : module en fonction du temps échantillonné
plot(discr, Mod(fftc), type = "l", xlab = "temps (ms)", ylab = "|FFT|", main = "Module de la FFT")
```

Module de la FFT



```
# Second tracé : module en fonction de la fréquence
# (0:(length(fftc)-1))/0.002 correspond au vecteur des fréquences en Hz (période totale 0.002 s)
plot((0:(length(fftc) - 1)) / 0.002, Mod(fftc), type = "l", xlab = "fréquence (Hz)", ylab = "|FFT|", ma
```

Analyse fréquentielle



1.4 5.6 Filtrage du signal

Pour éliminer le bruit du signal, on neutralise les composantes de la FFT dont le module est inférieur à un seuil (ici 5), puis on reconstitue le signal filtré avec l'inverse de la FFT.

```
# Copie des coefficients FFT pour filtrage
fftf <- fftc

# Mettre à zéro les composantes de petite amplitude (seuil = 5)
fftf[Mod(fftf) < 5] <- 0

# Reconstruction du signal filtré par la transformée inverse
signalf <- fft(fftf, inverse = TRUE) / length(discr)

# Tracer le signal filtré et le signal analogique pour comparaison
plot(discr, signalf, type = "l", col = "red", xlab = "temps (ms)", ylab = "amplitude", main = "Signal f
lines(discrn, signaln, col = "black")
legend("topright", legend = c("Signal filtré", "Signal analogique"), col = c("red", "black"), lty = 1)
```

Signal filtré vs signal analogique

